# Chapter 8

# Understanding an RDBMS

According to its official web site, MySQL is "the world's most popular open-source database." That's no small claim, but the numbers seem to back it up: today, over five million sites are creating, using, and deploying MySQL or MySQL-based applications. There are numerous reasons for this popularity: the server is fast and scalable, offers all the features and reliability of commercial-grade competitors, comes with a customer-friendly licensing policy, and is simple to learn and use. It's also well suited for development—the PHP programming language has supported MySQL since its early days, and the PHP-MySQL combination has become extremely popular for building database-driven web applications.

The previous chapters showed you the basics of PHP scripting, with discussions of PHP syntax and examples of common techniques you'll use when building PHP-based applications. This chapter focuses on the other half of the PHP-MySQL combo, giving you a crash course in basic RDBMS concepts and introducing you to the MySQL command-line client. In case you've never used a database before or the thought of learning another language scares you, don't worry, because MySQL is quite friendly and you should have no trouble learning how to use it.

## How to…

- Organize data into fields, records, and tables
- Identify records uniquely with primary keys
- Connect records in different tables through common fields
- Understand the three components of Structured Query Language (SQL)
- Write simple SQL statements
- Gain the benefits of normalized databases
- Send commands to, and receive responses from, MySQL with the command-line MySQL client

# Understanding a Relational Database

You may remember from the introductory notes in Chapter 1 that an electronic database management system (DBMS) is a tool that helps you organize information efficiently, so it becomes easier to find exactly what you need. A relational database

management system (RDBMS) like MySQL takes things a step further, by enabling you to create links between the various pieces of data in a database, and then use the relationships to analyze the data in different ways.

Now, while this is wonderful theory, it is still just that: theory. To truly understand how a database works, you need to move from abstract theoretical concepts to practical real-world examples. This section does just that, by creating a sample database and using it to explain some of the basic concepts you must know before proceeding further.

## Understanding Tables, Records, and Fields

Every database is composed of one or more *tables.* These tables, which structure data into rows and columns, are what lend organization to the data.

Here's an example of what a typical table looks like:



```
+-----+--------------------+-------+
| mid | mtitle             | myear |
+-----+--------------------+-------+
|   1 | Rear Window        |  1954 |
|   2 | To Catch A Thief   |  1955 |
|   3 | The Maltese Falcon |  1941 |
|   4 | The Birds          |  1963 |
|   5 | North By Northwest |  1959 |
|   6 | Casablanca         |  1942 |
|   7 | Anatomy Of A Murder |  1959 |
+-----+--------------------+-------+
```

As you can see, a table divides data into rows, with a new entry (or *record*) on every row. The data in each row is further broken down into columns (or *fields*), each of which contains a value for a particular attribute of that data. For example, if you consider the record for the movie *Rear Window,* you'll see that the record is clearly divided into separate fields for the row number, the movie title, and the year in which it was released.

TIP

*Think of a table as a drawer containing files. A record is the electronic representation of a file in the drawer.*

## Understanding Primary and Foreign Keys

Records within a table are not arranged in any particular order—they can be sorted alphabetically, by ID, by member name, or by any other criteria you choose

to specify. Therefore, it becomes necessary to have some method of identifying a specific record in a table. In the previous example, each record is identified by a unique number and this unique field is referred to as the *primary key* for that table. Primary keys don't appear automatically; you have to explicitly mark a field as a primary key when you create a table.


*Think of a primary key as a label on each file, which tells you what it contains. In the absence of this label, the files would all look the same and it would be difficult for you to identify the one(s) you need.*

With a relational database system like MySQL, it's also possible to link information in one table to information in another. When you begin to do this, the true power of an RDBMS becomes evident. So let's add two more tables, one listing important actors and directors, and the other linking them to movies.

```
+-----+--------------------+------+------------+
| pid | pname              | psex | pdob       |
+-----+--------------------+------+------------+
|   1 | Alfred Hitchcock   | M    | 1899-08-13 |
|   2 | Cary Grant         | M    | 1904-01-18 |
|   3 | Grace Kelly        | F    | 1929-11-12 |
|   4 | Humphrey Bogart    | M    | 1899-12-25 |
|   5 | Sydney Greenstreet | M    | 1879-12-27 |
|   6 | James Stewart      | M    | 1908-05-20 |
+-----+--------------------+------+------------+
```

```
+-----+-----+------+
| mid | pid | role |
+-----+-----+------+
|   1 |   1 | D    |
|   1 |   3 | A    |
|   1 |   6 | A    |
|   2 |   1 | D    |
|   2 |   2 | A    |
|   2 |   3 | A    |
|   3 |   4 | A    |
|   3 |   5 | A    |
|   4 |   1 | D    |
|   5 |   1 | D    |
|   5 |   2 | A    |
|   6 |   4 | A    |
+-----+-----+------+
```

Did you know?

## Invasion of the Foreign Keys

Referential integrity is a basic concept with an RDBMS, and one that becomes important when designing a database with more than one table. When foreign keys are used to link one table to another, referential integrity, by its nature, imposes constraints on inserting new records and updating existing records. For example, if a table only accepts certain types of values for a particular field, and other tables use that field as their foreign key, this automatically imposes certain constraints on the dependent tables. Similarly, referential integrity demands that a change in the field used as a foreign key—a deletion or new insertion—must immediately be reflected in all dependent tables.

Many of today's databases take care of this automatically—if you've worked with Microsoft Access, for example, you'll have seen this in action—but some don't. In the case of the latter, the task of maintaining referential integrity becomes a manual one, in which the values in all dependent tables have to be manually updated whenever the value in the primary table changes. Because using foreign keys can degrade the performance of your RDBMS, MySQL leaves the choice of activating such automatic updates (and losing some measure of performance) or deactivating foreign keys (and gaining the benefits of greater speed) to the developer, by making it possible to choose a different type for each table.



If you take a close look at the third table, you'll see that it links each movie with the people who participated in it, and it also indicates if they were actors (`A`) or directors (`D`). Thus, you can see that *Rear Window* (movie #1) was directed by Alfred Hitchcock (person #1), with Grace Kelly (person #3) and James Stewart (person #6) as actors. Similarly, you can see that Cary Grant (person #2) acted in two movies in the list, *To Catch A Thief* (movie #2) and *North By Northwest* (movie #5).

To understand these relationships visually, look at Figure 8-1.

The third table sets up a relationship between the first and second table, by linking them together using common fields. Such relationships form the foundation of a relational database system. The common fields used to link the tables together

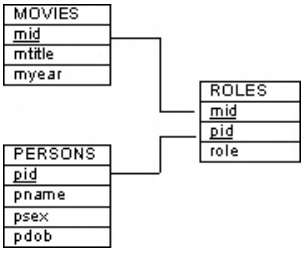


FIGURE 8-1 The interrelationships among movies, actors, and directors

are called *foreign keys,* and when every foreign key value is related to a field in another table, this relationship being unique, the system is said to be in a state of *referential integrity.* In other words, if the `mid` field is present once and only once in each table that uses it, and if a change to the `mid` field in any single table is reflected in all other tables, referential integrity is said to exist.

Once one or more relationships are set up between tables, it is possible to extract a subset of the data (a *data slice*) to answer specific questions. The act of pulling out this data is referred to as a *query,* and the resulting data is referred to as a *result set.* And it's in creating these queries, as well as in manipulating the database itself, that SQL truly comes into its own.

# Understanding SQL and SQL Queries

Putting data into a database is only half the battle—the other half involves using it effectively. This section tells you a little bit about SQL, which is the primary means of communicating with a database and extracting the data you require.

SQL began life as SEQUEL, the Structured English Query Language; the name was later changed to SQL for legal reasons. SEQUEL was a part of System/R, a prototype of the first relational database system created by IBM in 1974. In the late 1970s, SQL was selected as the query language for the Oracle RDBMS. This put it on the map and, by the 1980s, SQL was used in almost all commercial RDBMS. In 1989, SQL became an ANSI standard. The latest version of this standard, referred to as SQL92 or SQL2, is currently used on most of today's commercial RDBMSs (including MySQL).

SQL statements resemble spoken English and can broadly be classified into three categories:

- **Data Definition Language (DDL)** DDL consists of statements that define the structure and relationships of a database and its tables. Typically, these statements are used to create, delete, and modify databases and tables; specify field names and types; set indexes; and establish relationships between tables.
- **Data Manipulation Language (DML)** DML statements are related to altering and extracting data from a database. These statements are used to add records to, and delete records from, a database; perform *queries;* retrieve table records matching one or more user-specified criteria; and join tables together using their common fields.
- **Data Control Language (DCL)** DCL statements are used to define access levels and security privileges for a database. You would use these statements to grant or deny user privileges, assign roles, change passwords, view permissions, and create rulesets to protect access to data.




*When creating applications with PHP and MySQL, you'll mostly be using DML statements.*

Here are a few examples of valid SQL statements:

```
CREATE DATABASE addressbook;
DESCRIBE catalog;
SELECT title FROM books WHERE targetAge > 3;
DELETE FROM houses WHERE area < 100;
```

As the previous examples demonstrate, SQL syntax is close to spoken English, which is why most novice programmers find it easy to learn and use. Every SQL statement begins with an "action word" and ends with a semicolon. White space, tabs, and carriage returns are ignored. This makes the following two commands equivalent:

```
DELETE FROM houses WHERE monthlyRent > 25000;
DELETE FROM
      houses
WHERE monthlyRent >

25000;
```

# Understanding Database Normalization

An important part of designing a database is a process known as normalization. *Normalization* refers to the activity of streamlining a database design by eliminating redundancies and repeated values. Most often, redundancies are eliminated by placing repeating groups of values into separate tables and linking them through foreign keys. This not only makes the database more compact and reduces the disk space it occupies, but it also simplifies the task of making changes. In nonnormalized databases, because values are usually repeated in different tables, altering them is a manual (and error-prone) find-and-replace process. In a normalized database, because values appear only once, making changes is a simple one-step `UPDATE`.

The normalization process also includes validating the database relationships to ensure that there aren't any crossed wires and to eliminate incorrect dependencies. This is a worthy goal, because when you create convoluted table relationships, you add greater complexity to your database design … and greater complexity translates into slower query time as the optimizer tries to figure out how best to handle your table joins.

A number of so-called normal forms are defined to help you correctly normalize a database. A *normal form* is simply a set of rules that a database must conform to. Five such normal forms exist, ranging from the completely nonnormalized database to the fully normalized one.


*To see an example of how to go about turning a badly designed database into a well-designed one, visit Chapter 12, or look online at* ***http://dev.mysql.com/tech-resources/articles/intro-to-normalization.html*** *for a primer on the topic.*

# Using the MySQL Command-Line Client

The MySQL RDBMS consists of two primary components: the MySQL database server itself, and a suite of client-side programs, including an interactive client and utilities to manage MySQL user permissions, view and copy databases, and import and export data. If you installed and tested MySQL according to the procedure outlined in Chapter 2 of this book, you've already met the MySQL command-line client. This client is your primary means of interacting with the MySQL server and, in this section, I'll be using it to demonstrate how to communicate with the server.

To begin, ensure that your MySQL server is running, and then connect to it with the `mysql` command-line client. Remember to send a valid password with your username, or else MySQL will reject your connection attempt. (Throughout this section and the ones that follow, boldface type is used to indicate commands that you should enter at the prompt.)

```
[user@host]# mysql -u root -p
Password: ******
```

If all went well, you'll see a prompt like this:

```
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 134 to server version: 4.0.12
Type 'help;' or '\h' for help. Type '\c' to clear the buffer.
mysql>
```



The `mysql>` you see is an interactive prompt, where you enter SQL statements. Statements entered here are transmitted to the MySQL server using a proprietary client-server protocol, and the results are transmitted back using the same manner.

Try this out by sending the server a simple statement:

```
mysql> SELECT 5+5;
+-----+
| 5+5 |
+-----+
|  10 |
+-----+
1 row in set (0.06 sec)
```

Here, the `SELECT` statement is used to perform an arithmetic operation on the server and return the results to the client (you can do a lot more with the `SELECT` statement, and it's all covered in Chapter 10). Statements entered at the prompt must be terminated with either a semicolon or a `\g` signal, followed by a carriage return to send the statement to the server. Statements can be entered in either uppercase or lowercase type.

The response returned by the server is displayed in tabular form, as rows and columns. The number of rows returned, as well as the time taken to execute the command, are also printed. If you're dealing with extremely large databases, this information can come in handy to analyze the speed of your queries.

Did you know?

## Version Control

Different MySQL server versions support different functions. MySQL 3.*x* included support for joins; MySQL 4.*x* added support for transactions; MySQL 4.1.*x* introduced subqueries, prepared statements and multiple character sets; and the upcoming MySQL 5.*x* promises to support views, stored procedures, and triggers. To find out which version of the MySQL server you're running, look in the message text displayed by the client when it first connects, or use the `SELECT VERSION()` command.

As noted previously, white space, tabs, and carriage returns in SQL statements are ignored. In the MySQL command-line client, typing a carriage return without ending the statement correctly simply causes the client to jump to a new line and wait for further input. The continuation character `->` is displayed in such situations to indicate that the statement is not yet complete. This is illustrated in the next example, which splits a single statement over three lines:

```
mysql> SELECT 100
    -> *
    -> 9 + (7*2);
+-----------------+
| 100
*
9 + (7*2)         |
+-----------------+
|             914 |
+-----------------+
1 row in set (0.00 sec)
```

Notice that the SQL statement in the previous example is only transmitted to the server once the terminating semicolon is entered.

Most of the time, you'll be using SQL to retrieve records from one or more MySQL tables. Consider, for example, the following simple SQL query, which

counts all the records in the `user` table from the `mysql` database (this database comes preinstalled with MySQL):

```
mysql> SELECT COUNT(*) FROM mysql.user;
+----------+
| COUNT(*) |
+----------+
|        4 |
+----------+
1 row in set (0.11 sec)
```

To obtain help on using the MySQL client, type **help** at the `mysql>` prompt.

```
mysql> help
List of all MySQL commands:
   (Commands must appear first on line and end with ';')

help    (\h)   Display this help.
?       (\?)   Synonym for `help'.
clear   (\c)   Clear command.
connect (\r)   Reconnect to the server. Optional arguments are db and host.
ego     (\G)   Send command to mysql server, display result vertically.
exit    (\q)   Exit mysql. Same as quit.
go      (\g)   Send command to mysql server.
...
```

To close the connection to the server and exit the client, type **quit** at the `mysql>` prompt.

```
mysql> quit
Bye
```

# Interacting with MySQL Through a Graphical Client

If using the command line isn't your style, don't lose heart—a number of good graphical clients also exist for MySQL. Here's a list of the better ones:

- **MySQL Control Center (http://www.mysql.com/products/mysqlcc/index.html)** is an excellent front-end query and database management tool for MySQL. Currently, Windows, UNIX, and Linux versions are available, with a Mac OS X version under development.

- **SQLyog (http://www.webyog.com/sqlyog/)** is a Windows-based front-end for MySQL administration. It offers a graphical interface that supports copying and pasting query results, syntax highlighting, and datagrid display, and it includes a synchronization tool to synchronize databases on different servers.
- **phpMyAdmin (http://www.phpmyadmin.net/)** is a web-based tool to manage MySQL databases and tables, input records, and execute queries. It is written entirely in PHP, is licensed under the GNU GPL, and is currently available in 47(!) languages.

# Summary

This chapter focused on getting you started with MySQL, by teaching you the basic concepts you need to know to use MySQL efficiently. It showed you how a database structures data into tables, records, and fields; how it identifies records with primary keys; and how it connects records in different tables with each other through foreign keys. This chapter also introduced you to SQL, giving you a brief look at some SQL commands—these commands will be discussed in greater detail over the next few chapters. Finally, an introduction to database normalization and a few examples of using the MySQL command-line client wrapped things up.

If you're interested in learning more about the topics in this chapter, these web links have more information:

- The official MySQL tutorial, at **http://dev.mysql.com/doc/mysql/en/Tutorial.html**
- The origins of MySQL, at **http://dev.mysql.com/doc/mysql/en/History.html**
- A detailed discussion of basic RDBMS concepts, at **http://www.melonfire.com/community/columns/trog/article.php?id=52**
- The normal forms of a database design, at **http://en.wikipedia.org/wiki/Database_normalization**
- The MySQL command-line client, at **http://dev.mysql.com/doc/mysql/en/Client-Side_Scripts.html**